REPUBLIC OF CAMEROON

****

PEACE-WORK-FATHERLAND

****

REPUBLIQUE DU CAMEROUN

****

PAIX-TRAVAIL- PATRIE

****

Course: C Programming

PKFOKAM INSTITUTE OF EXCELLENCE

PROJECT: WEDDING INVITATION AND GIFT MANAGEMENT SYSTEM

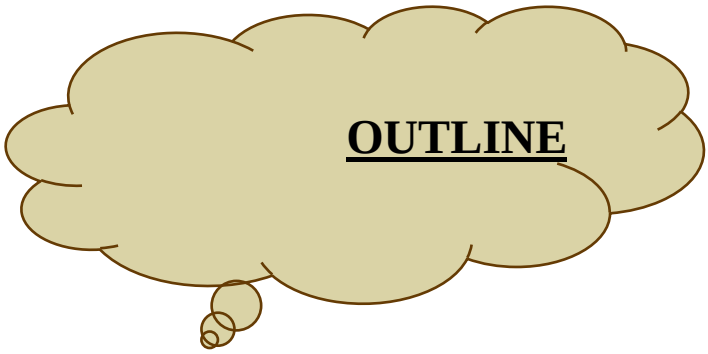
PROJECT REPORT 1

GIT HUB REPOSITORY AND PERSON MANAGEMENT CREATION

Framework	Django (Python 3.x)
Database	MySQL Restaurant DB
Architecture	MVT (Model-View-Template)
Modules	5 functional modules
Team Size	5 students

Group Members	Participation	Matricules
Talla Tchossi Daniella Elsa	20%	26S018
Djomo Yepdi Evans	20%	26S001
Matchim Dzuyim Verenne Celia	20%	26S031
Mekolle Tuatsop Dalhia Aure	20%	26S004
Messi Zang Paul Jacques	20%	26S005

Lecturer: Mr. Mbietieu Amos
Mb ieutieu
Academic Year: 202 5_202 6



OUTLINE

1. Project Overview

The **Wedding Invitation and Gift Management System (WIGMS)** is a software application designed to assist in organizing and managing wedding events. Weddings in Cameroon often involve a large number of guests, gift contributions, and multiple social categories, which makes manual management difficult.

The purpose of this system is to simplify wedding planning by providing tools to manage:

- Guest information
- Social categories of guests
- Invitation priority
- Wedding gift tracking

The application is developed using the **C programming language** and follows a **modular programming approach** where each part of the system is implemented as an independent module.

The system aims to improve organization, reduce errors, and help wedding planners efficiently manage guest lists and gifts.

2. Development Tools Used

The following tools were used during the development of the system:

Tool	Purpose
C Programming Language	Main programming language used for system implementation
GCC Compiler	Used to compile and run the C programs
Visual Studio Code / CodeBlocks	Code editor for writing and debugging the program
GitHub	Version control and collaboration platform

Standard C Libraries (stdio.h, ++ stdlib.h, string.h)	Used for input/output operations, memory management and string handling

These tools help ensure efficient development, collaboration, and proper project management.

3. Work Organisation and task distribution

The group consisted of 5 members. The work organisation depended on assigning responsibilities with respect to skills and the project phases. This approach permitted us to ensure that each section of this report would benefit from dedicated expertise.

Group Member	Main Role	Assigned Tasks
1. Matchim Dzuyim Verenne Celia	Implementation and testing	**Common tasks:** Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. **Individual tasks:** GTK, CSV, Test of Person Management, Test of Category Management.
2. Mekolle Tuatsop Dalhia Aure	Documentation and planning	**Common tasks:** Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. **Individual tasks:** Report 1, Gantt Project, Report 2, Implementation of Category Management.
3. Talla Tchossi Daniella Elsa	Development (core modules)	**Common tasks:** Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. **Individual tasks:** GTK, CSV, Implementation of Person Management.
4. Messi Zang Paul Jacques	System design (UML diagrams)	**Common tasks:** Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. **Individual tasks:** Use Case Diagram, Class Diagram.
5. Djomo Yepdi Evans	System modeling and deployment	**Common tasks:** Project Overview, Development Tools Used, System Architecture, Phase 1 Summary. **Individual tasks:** Sequence Diagram, Deployment Diagram.

4. System Architecture

The application follows a **simplified modular architecture inspired by the MVC design pattern**. The architecture consists of three main components:

- **Model**

The model contains the core data structures and logic of the program, including:

- Person management
- Category management
- Priority management
- Gift management

- **Controller**

The controller handles the flow of the application by managing the interactions between the user interface and the program modules.

- **View**

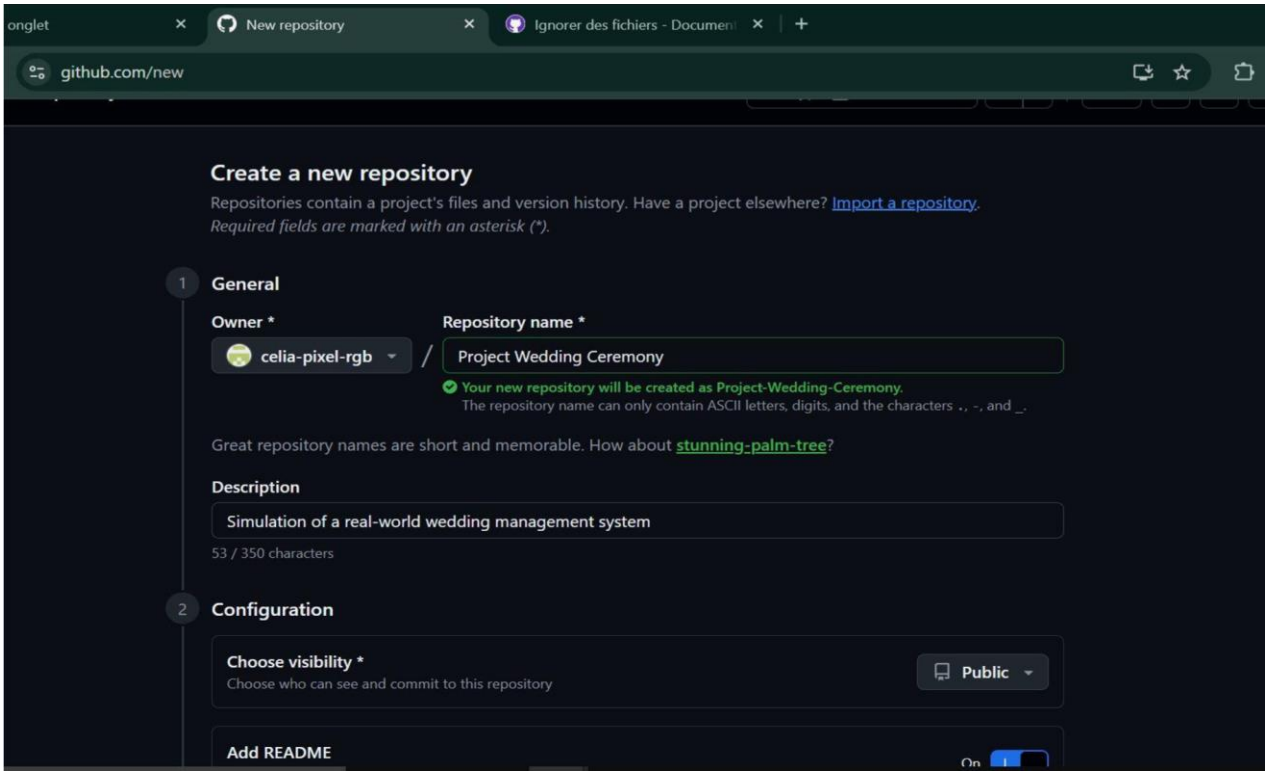
The view represents the interface presented to the user. In this project, the view is implemented through a **console-based menu interface** that allows the user to navigate through the different system functionalities.

This architecture ensures clear separation between data, logic, and user interaction.

5. Functional Modules of the Application

As first stage, we did what we consider as our Module 1: GitHub creation + Personal Management. This module focuses on:

Creating GitHub repository

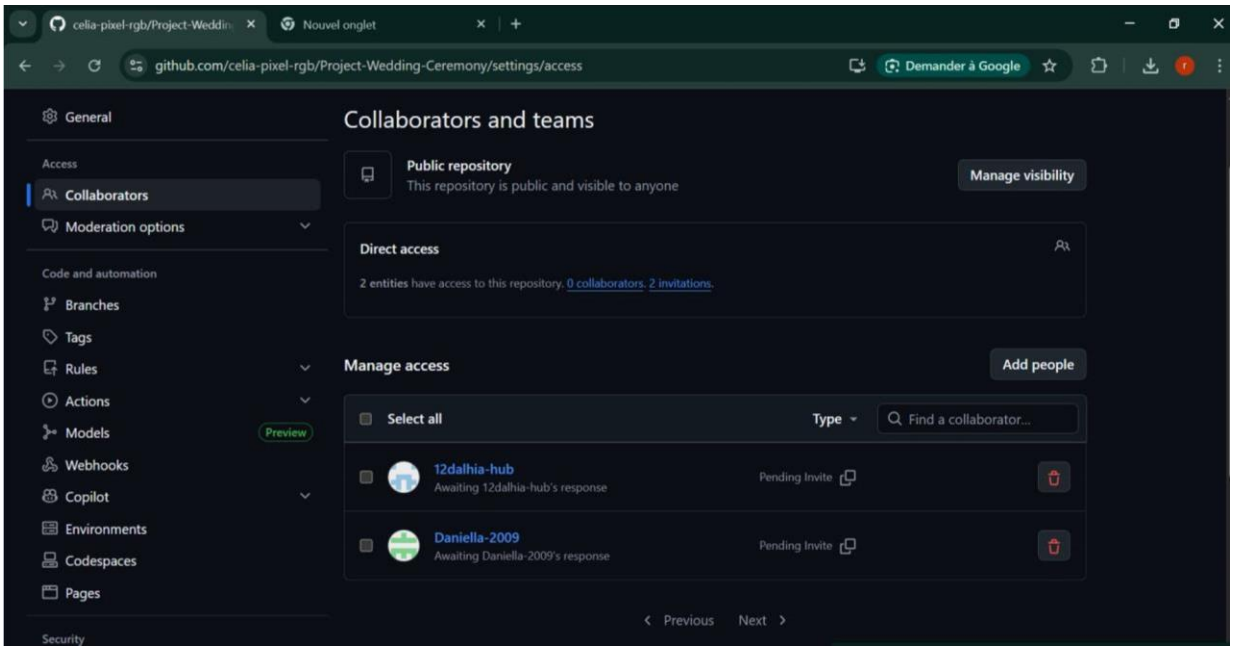


The group leader created the public repository with the following settings:
Here is the GitHub link to access our repository:
https://github.com/celia-pixel-rgb/Project-Wedding-Ceremony

SETTINGS	VALUE CHOSEN
Repository name	Project-Wedding-Ceremony
Visibility	Public
.gitignore	C template selected
Repository link	https://github.com/celia-pixelrgb/Project-Wedding-Ceremony

Adding Collaborators

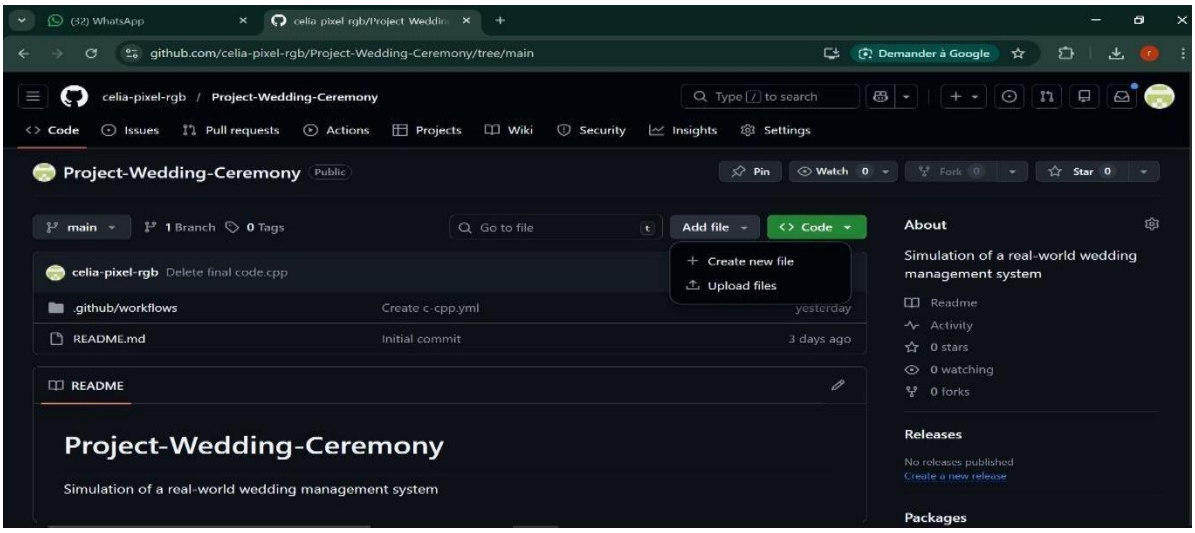
The remaining group members were added as collaborators through the Setting --» collaborators page of the repository. Each member received an Email invitation from GitHub which they accepted in order to gain push access to the repository.

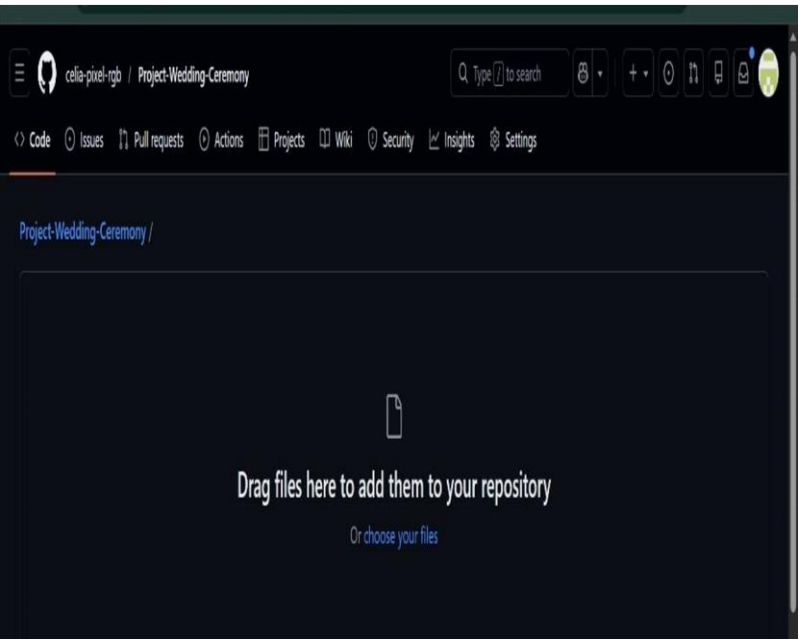


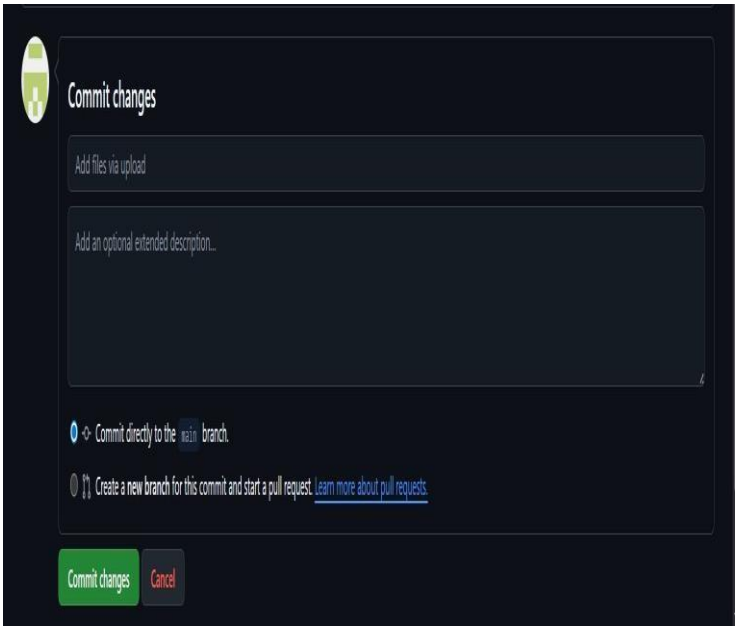
Cloning the Repository and first Push

After accepting their invitation, each member cloned the depository into their local machine. The group leader then performed the first push of the program.

```
Microsoft Windows [version 10.0.17763.1577]
(c) 2018 Microsoft Corporation. Tous droits réservés.

C:\Users\user>git clone https://github.com/celia-pixel-rgb/Project-Wedding-Ceremony.git
Cloning into 'Project-Wedding-Ceremony'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

C:\Users\user>
```







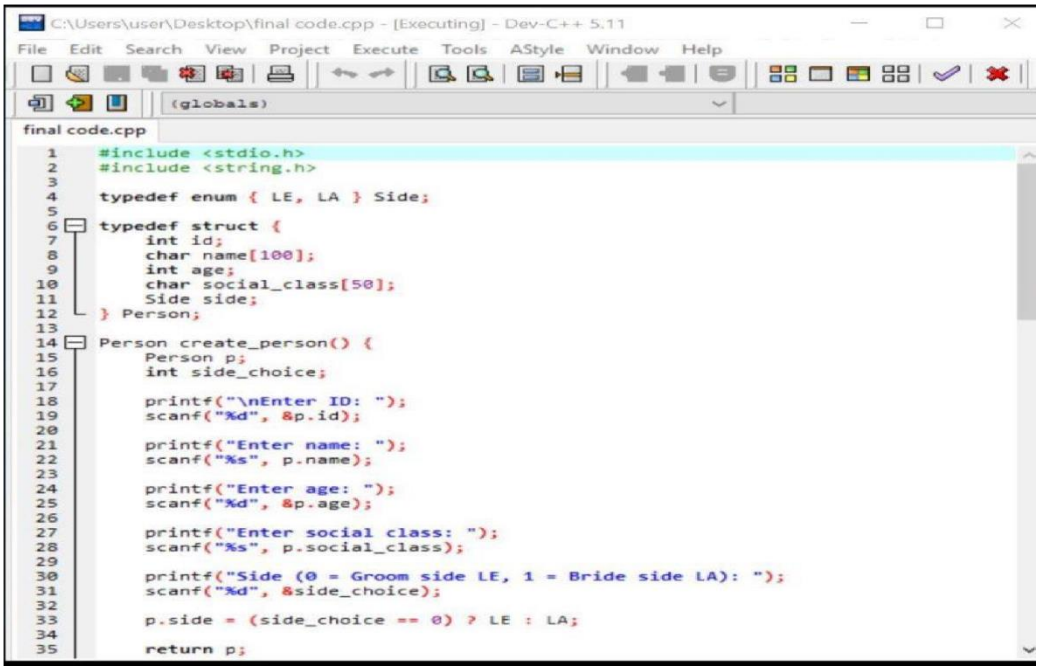
Creation of the c program to manage the guests

This module manages guest information. It allow the system to:

- **Create a new guest**

Here, we are creating a program that can create persons (guests). Based on the wanted total number of guests entered by the user, we defined a structure in which we register guests base on their name, age, Id, social status and their side (either the bride or the groom).

-



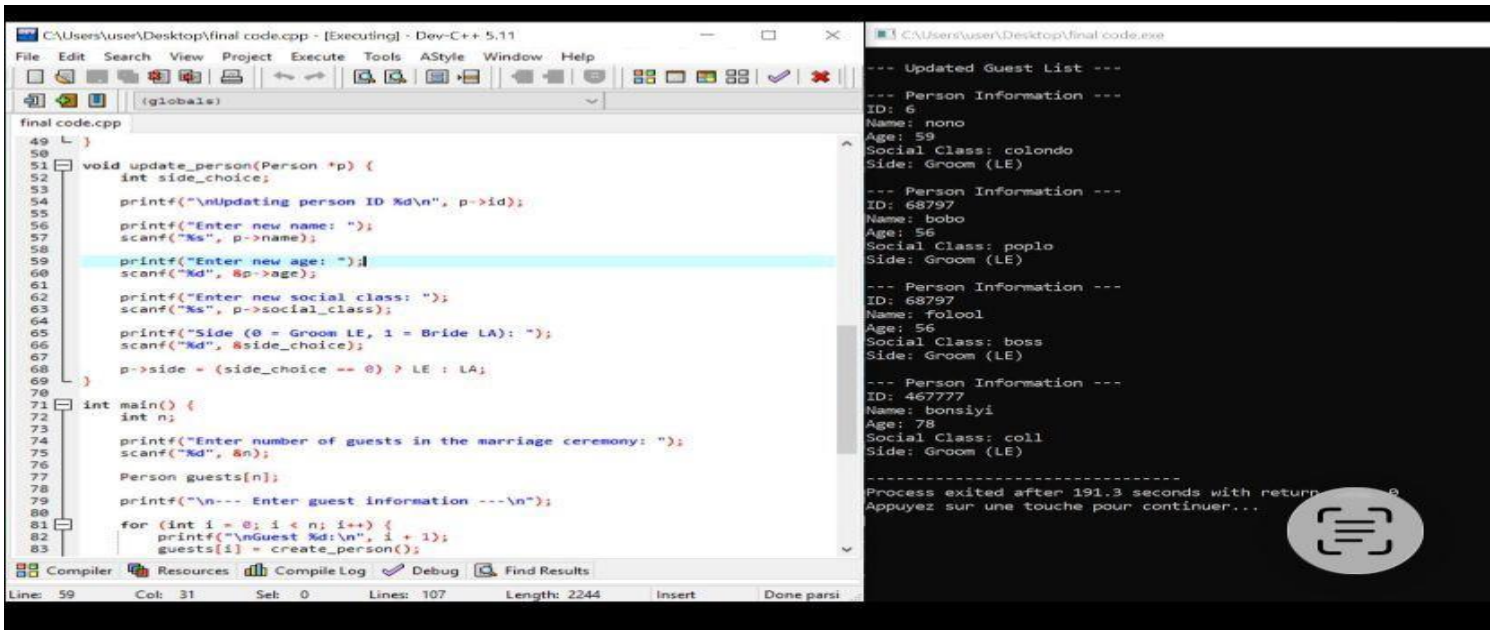
```
#include <stdio.h>
#include <string.h>

typedef enum { LE, LA } Side;

typedef struct {
    int id;
    char name[100];
    int age;
    char social_class[50];
    Side side;
} Person;

Person create_person() {
    Person p;
    int side_choice;

    printf("\nEnter ID: ");
    scanf("%d", &p.id);

    printf("Enter name: ");
    scanf("%s", p.name);

    printf("Enter age: ");
    scanf("%d", &p.age);

    printf("Enter social class: ");
    scanf("%s", p.social_class);

    printf("Side (0 = Groom side LE, 1 = Bride side LA): ");
    scanf("%d", &side_choice);

    p.side = (side_choice == 0) ? LE : LA;

    return p;
```

Display the guest

-

```
void display_person(Person p) {
    printf("\n--- Person Information ---\n");
    printf("ID: %d\n", p.id);
    printf("Name: %s\n", p.name);
    printf("Age: %d\n", p.age);
    printf("Social Class: %s\n", p.social_class);

    if (p.side == LE)
        printf("Side: Groom (LE)\n");
    else
        printf("Side: Bride (LA)\n");
}
```

Update Guest



```
}
void update_person(Person *p) {
    int side_choice;

    printf("\nUpdating person ID %d\n", p->id);

    printf("Enter new name: ");
    scanf("%s", p->name);

    printf("Enter new age: ");
    scanf("%d", &p->age);

    printf("Enter new social class: ");
    scanf("%s", p->social_class);

    printf("Side (0 = Groom LE, 1 = Bride LA): ");
    scanf("%d", &side_choice);

    p->side = (side_choice == 0) ? LE : LA;
}
int main() {
    int n;

    printf("Enter number of guests in the marriage ceremony: ");
    scanf("%d", &n);

    Person guests[n];

    printf("\n--- Enter guest information ---\n");

    for (int i = 0; i < n; i++) {
        printf("\nGuest %d:\n", i + 1);
        guests[i] = create_person();
```

```
--- Updated Guest List ---

--- Person Information ---
ID: 6
Name: nono
Age: 59
Social Class: colondo
Side: Groom (LE)

--- Person Information ---
ID: 68797
Name: bobo
Age: 56
Social Class: poplo
Side: Groom (LE)

--- Person Information ---
ID: 68797
Name: folool
Age: 56
Social Class: boss
Side: Groom (LE)

--- Person Information ---
ID: 467777
Name: bonsiyi
Age: 78
Social Class: coll
Side: Groom (LE)

--------------------------------
Process exited after 191.3 seconds with return
Appuyez sur une touche pour continuer...
```

6. Main Difficulties Encountered

During the development of the project, several challenges were encountered:

- Understanding the use of **structures in C programming**
- Correctly implementing **alternatives structures**, **loops and memory management**
- Designing linked list structures for dynamic data
- Managing user input validation
- Debugging errors during program execution

These challenges required careful testing and collaboration among team members.

7.Solutions Implemented

To address the difficulties encountered, the following solutions were implemented:

- Reviewing C programming documentation and tutorials
- Testing each function independently before integration
- Using debugging techniques to identify program errors
- Collaborating through GitHub to track and fix problems

These solutions allowed the team to successfully implement the first module of the project.

8. Phase 1 Summary

The first phase of the project was successfully completed.

The team was able to:

- Understand the project requirements
- Set up the development environment
- Create the project structure
- Implement the **Person Management module**
- Test the system functionality

The project is now ready to proceed to the next phase, which will involve implementing additional modules that we will present later on.

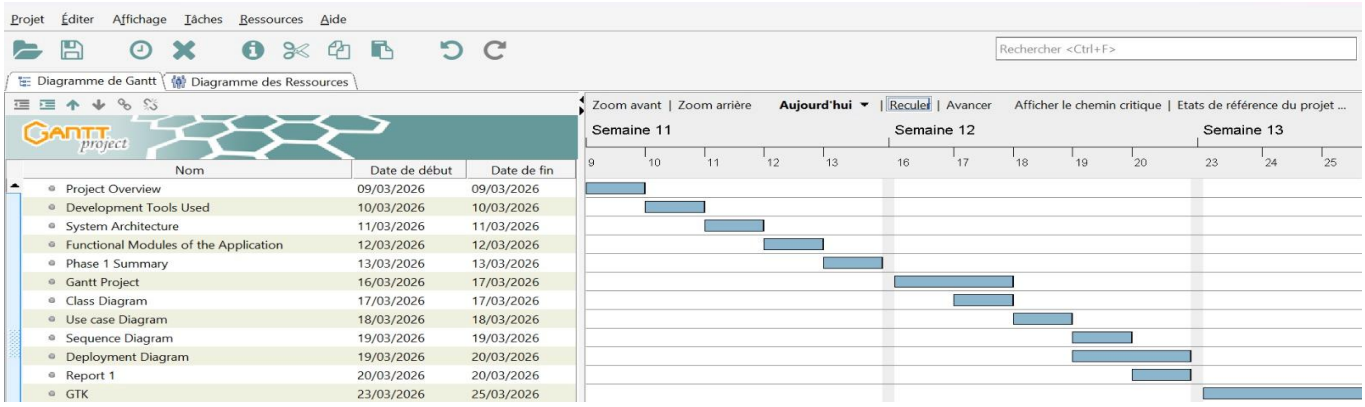
8. References

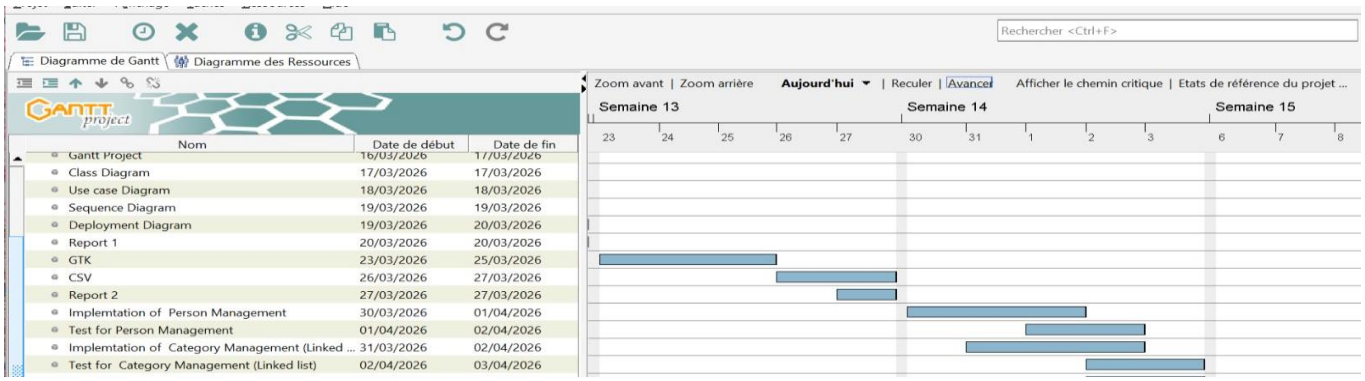
The following references were used during the development of this project:

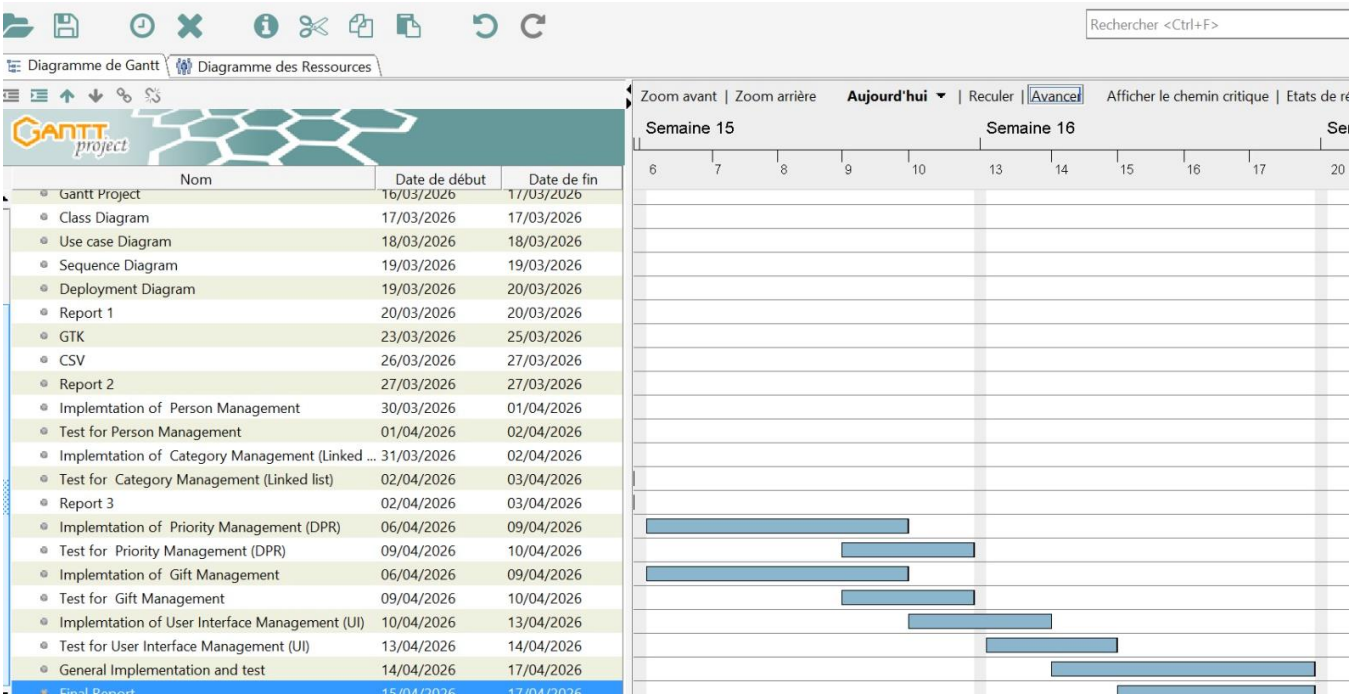
- **Brian W. Kernighan & Dennis M. Ritchie** – *The C Programming Language*
- **Stephen Prata** – *C Programming: A Modern Approach*
- **ISO C Programming Documentation**
- **GitHub Official Documentation** – https://docs.github.com
- Course materials provided by the lecturer

9.Gantt Project

This project outlines the planning, design, and implementation of a software system using structured project management techniques. The Gantt chart was used to organize tasks, timelines, and dependencies efficiently.







Phase		Key Activities	Outcome
Phase 1: Project Initiation & Planning		Project overview, identification of development tools, system architecture design	Clear understanding of project scope, tools, and technical structure
Phase 2: System Design		Functional modules definition, class diagram, use case diagram, sequence diagram, deployment diagram	Well-structured system design and interaction models
Phase 3: Project Documentation & Setup		Phase 1 summary, Gantt project creation	Organized documentation and project scheduling
Phase 4: Core Implementation		Implementation of person management, category management (linked list), priority management (DPR), gift management	Functional backend modules developed successfully
Phase 5: Testing & Validation		Testing of person, category, priority, and gift management modules	Identification and correction of system errors
Phase 6: User Interface Development		Implementation and testing of user interface (UI)	User-friendly interface ensuring system usability

Conclusion

The project was successfully structured using a Gantt chart, ensuring proper time management and task coordination.
All phases were completed systematically, resulting in a functional and well-tested application.

SYSTEM DESIGN

System design is a critical phase in software development that defines the structure, components, and interactions of a system. It provides a clear blueprint to guide the development process and ensure that all functional and technical requirements are met.

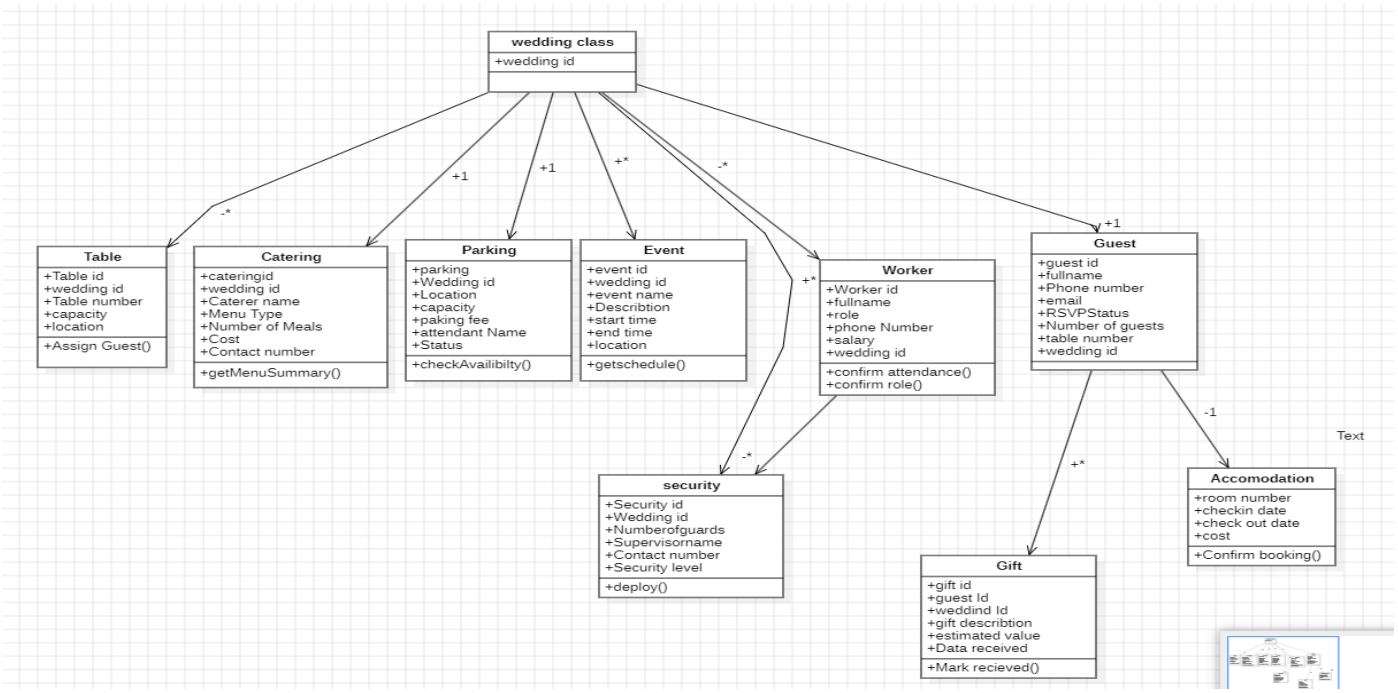
In this section, we will present and explain the key design diagrams used in the project, including the class diagram, sequence diagram, deployment diagram, and use case diagram, to illustrate how the system operates and how its components interact.

CLASS DIAGRAM

Overview

The WIGMS UML Class Diagram models wedding organization entities using StarUML. A central Wedding class connects nine supporting classes — each with detailed attributes and defined relationships via UML associations and multiplicity notation.

- A class diagram provides a static view of the system, showing classes, attributes, operations, and relationships.
- In WIGMS, it acts as a blueprint for implementation.
- The model contains ten main classes: Wedding, Guest, Worker, Security, Catering, Parking, Event, Table, Gift, and Accommodation. The Wedding class is the central hub linking all other elements.

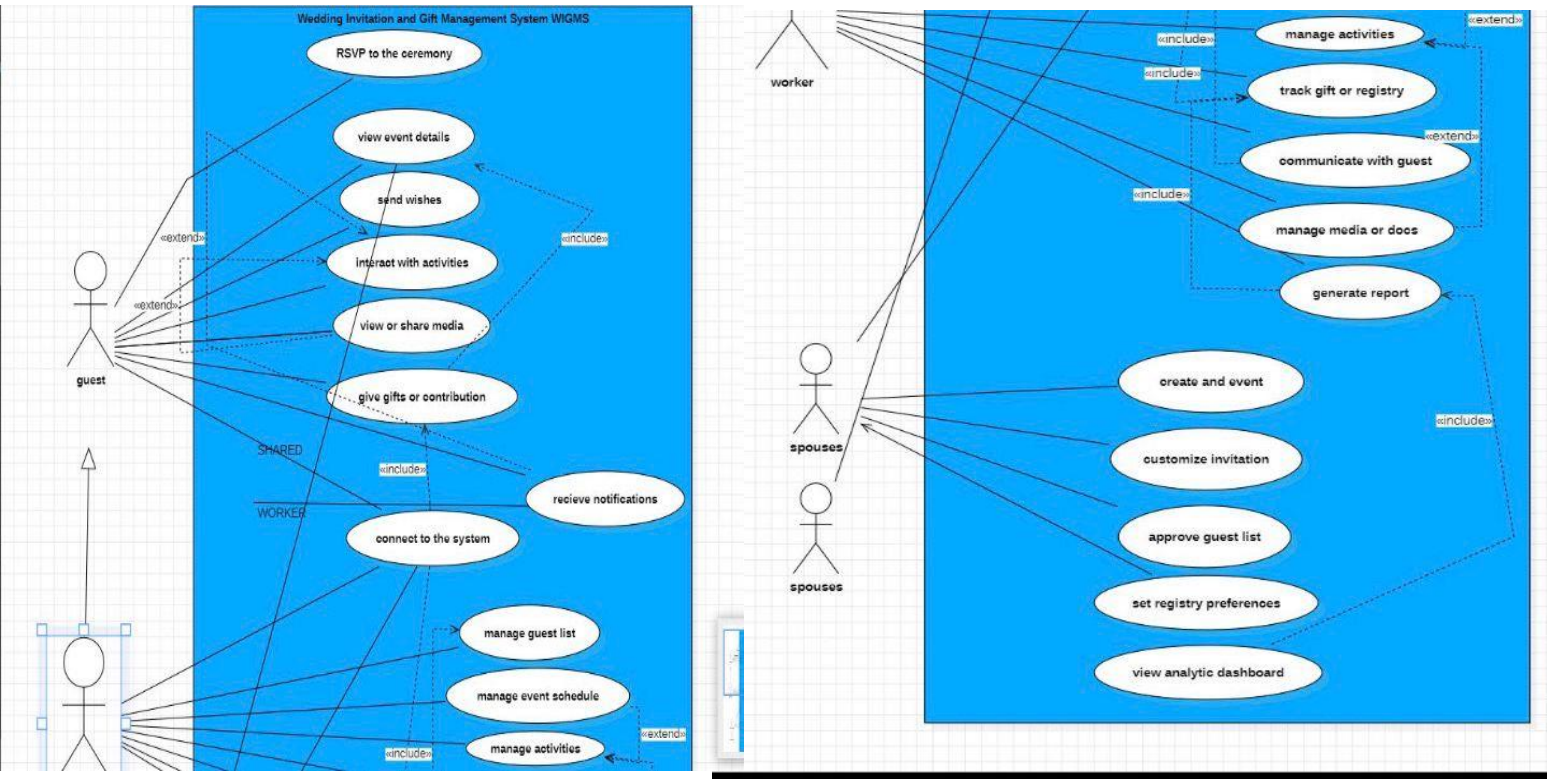


a. Class Diagram

USE CASE DIAGRAM

Overview

The WIGMS Use-Case Diagram models how users interact with the Wedding Invitation and Gift Management System. Designed in StarUML, it identifies two actors (Guest and Worker), maps their use cases, and defines system boundaries using «include» and «extend» relationships. The system reflects real-life wedding planning, particularly in the Yaoundé context.



b. Use-case Diagram

Actor	Role	Key Interactions
Guest	External wedding attendees.	RSVP, view event details, send wishes, give gifts, interact with activities, view/share media, receive notifications
Spouses	The wedding couple — primary system owners.	Create event, customise invitation, approve guest list, set registry preferences, view analytics, receive notifications
Worker	Event staff handling back-end operations.	Manage guest list, event schedule, activities, gifts, media/docs, communicate with guests, generate reports

SEQUENCE DIAGRAM

Overview

A sequence diagram illustrates how different components of the wedding-planning system interact over time through ordered message exchanges. It helps visualize the workflow of actions such as event creation, guest invitations, and RSVP tracking, ensuring processes occur in the correct sequence.

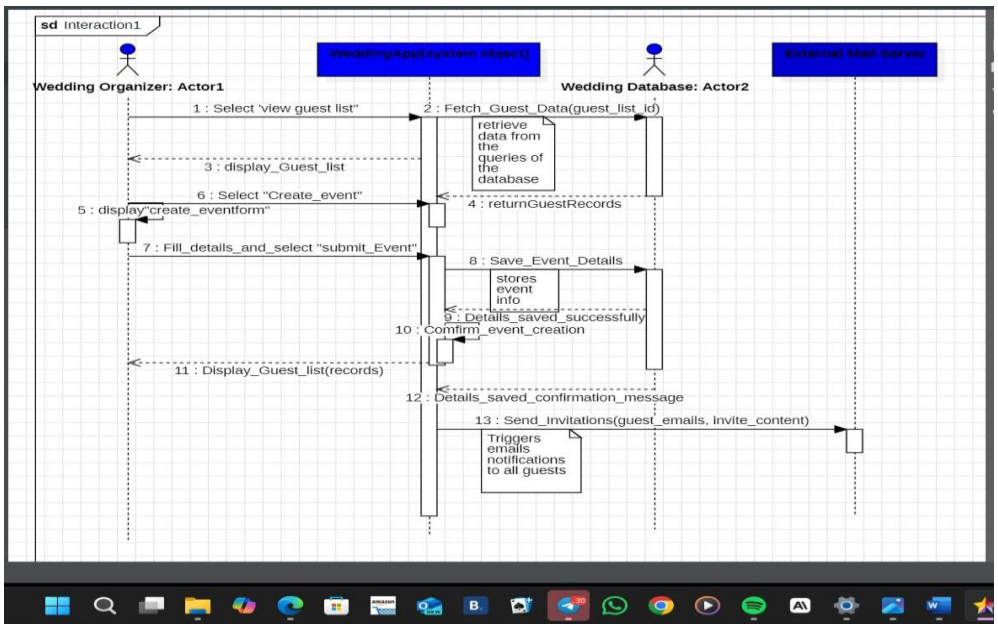


Figure 1: Sequence Diagram — Wedding Management System (StarUML)

DEPLOYMENT DIAGRAM

Deployment Diagram

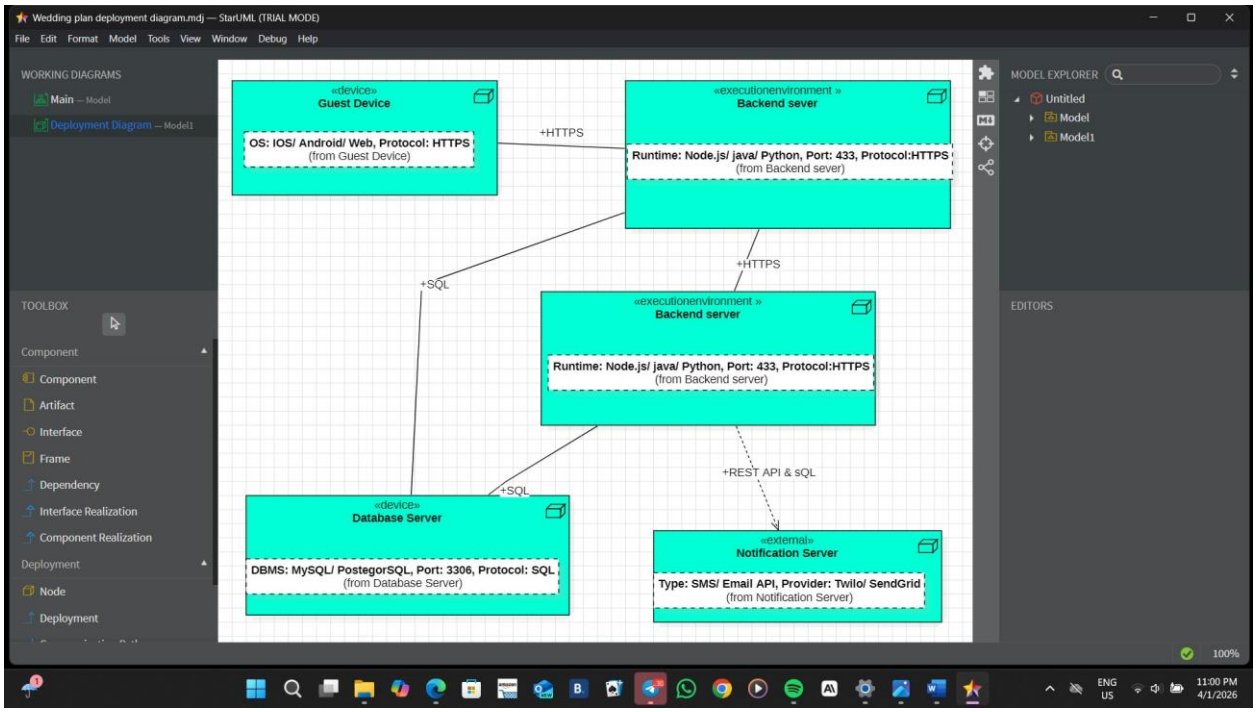


Figure 2: Deployment Diagram — Wedding Management System (StarUML)

1. **GNU Image Manipulation ProgramToolkit (GTK) and Comma-Selected Value (CSV**

GTK

Development of a GTK-based form application written in C for managing wedding guest data. The application collects user input through a graphical interface and stores it in a CSV file.

Program Overview

The program uses GTK4 to create a graphical interface consisting of input fields for Name, Age, Status, Phone, and Side. A Save button allows users to store the entered data into a CSV file named guests.csv.

Key Implementation Details

marriage.c

```c
#include <gtk/gtk.h>
#include <stdio.h>

// Input fields (global so we can access them easily)
GtkWidget *name_entry;
GtkWidget *age_entry;
GtkWidget *status_entry;
GtkWidget *phone_entry;
GtkWidget *side_entry;

// Function called when button is clicked
void save_data(GtkButton *button, gpointer user_data) {

    // Get text from input fields
    const char *name = gtk_editable_get_text(GTK_EDITABLE(name_entry));
    const char *age = gtk_editable_get_text(GTK_EDITABLE(age_entry));
    const char *status = gtk_editable_get_text(GTK_EDITABLE(status_entry));
    const char *phone = gtk_editable_get_text(GTK_EDITABLE(phone_entry));
    const char *side = gtk_editable_get_text(GTK_EDITABLE(side_entry));

    // Open CSV file
    FILE *file = fopen("guests.csv", "a");

    if (file == NULL) {
        g_print("Error opening file\n");
        return;
    }

    // Write data into file
    fprintf(file, "%s,%s,%s,%s,%s\n", name, age, status, phone, side);

    fclose(file);

    g_print("Data saved!\n");
}

// Function that creates the window
void activate(GtkApplication *app, gpointer user_data) {

    GtkWidget *window;
```

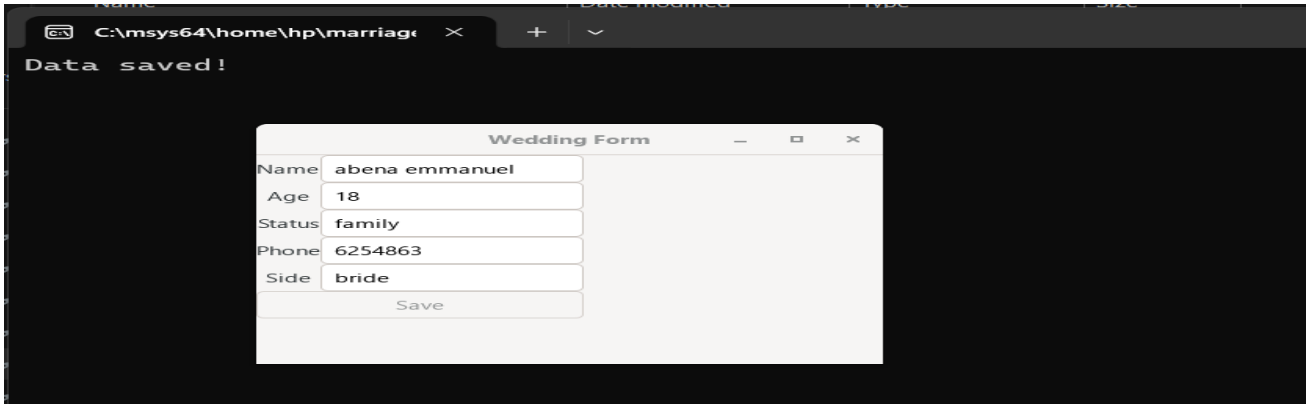
```c
// Function that creates the window
void activate(GtkApplication *app, gpointer user_data) {

    GtkWidget *window;
    GtkWidget *grid;
    GtkWidget *button;

    // Create window
    window = gtk_application_window_new(app);
    gtk_window_set_title(GTK_WINDOW(window), "Wedding Form");
    gtk_window_set_default_size(GTK_WINDOW(window), 400, 300);

    // Create layout
    grid = gtk_grid_new();
    gtk_window_set_child(GTK_WINDOW(window), grid);

    // Create input fields
    name_entry = gtk_entry_new();
    age_entry = gtk_entry_new();
    status_entry = gtk_entry_new();
    phone_entry = gtk_entry_new();
    side_entry = gtk_entry_new();

    // Add labels and fields
    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Name"), 0, 0, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), name_entry, 1, 0, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Age"), 0, 1, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), age_entry, 1, 1, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Status"), 0, 2, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), status_entry, 1, 2, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Phone"), 0, 3, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), phone_entry, 1, 3, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Side"), 0, 4, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), side_entry, 1, 4, 1, 1);

    // Create button
```

```c
    phone_entry = gtk_entry_new();
    side_entry = gtk_entry_new();

    // Add labels and fields
    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Name"), 0, 0, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), name_entry, 1, 0, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Age"), 0, 1, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), age_entry, 1, 1, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Status"), 0, 2, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), status_entry, 1, 2, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Phone"), 0, 3, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), phone_entry, 1, 3, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), gtk_label_new("Side"), 0, 4, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), side_entry, 1, 4, 1, 1);

    // Create button
    button = gtk_button_new_with_label("Save");
    gtk_grid_attach(GTK_GRID(grid), button, 0, 5, 2, 1);

    // Connect button click to function
    g_signal_connect(button, "clicked", G_CALLBACK(save_data), NULL);

    gtk_window_present(GTK_WINDOW(window));
}

// Main function
int main(int argc, char **argv) {
    GtkApplication *app;

    app = gtk_application_new("org.example.wedding", G_APPLICATION_DEFAULT_FLAGS);

    g_signal_connect(app, "activate", G_CALLBACK(activate), NULL);

    return g_application_run(G_APPLICATION(app), argc, argv);
}
```

- ✓ **Global widget pointers** were used to allow access to input fields across functions. This simplifies retrieving user input when the Save button is clicked.
- ✓ **The use of gtk_editable_get_text() allows** extraction of user input from each entry field. This is required because GTK widgets do not directly expose their content.
- ✓ **File handling** is implemented using fopen in append mode ("a").

  This ensures that new data is added without overwriting existing entries.

Csv file creation



The csv file was generated immediately after the GTK form worked, and was tested by inputing arbitrary entries

2. Problem Encountered and Solution

General Problem

- Difficulty in configuring development tools such as GTK.
- Limited understanding of UML diagrams (sequence, deployment, use case, class).
- Challenges in using StarUML (navigation, relationships, layout).
- Errors in coding such as file handling, compilation, and signal connections.
- Complexity in organizing system interactions and component communication.

General Solution

- Proper installation and configuration of development tools.
- Studying UML concepts and applying correct diagram rules.
- Practicing and exploring StarUML features for better usage.
- Applying debugging techniques and correct coding practices.
- Breaking the project into smaller tasks using structured planning tools like Gantt charts.

3. Conclusion

The Wedding Ceremony Management System demonstrates how C programming can be combined with project planning tools, system design diagrams, and user interface technologies. The use of GTK and CSV enhances usability and data management, making the system efficient and practical for real-world applications.

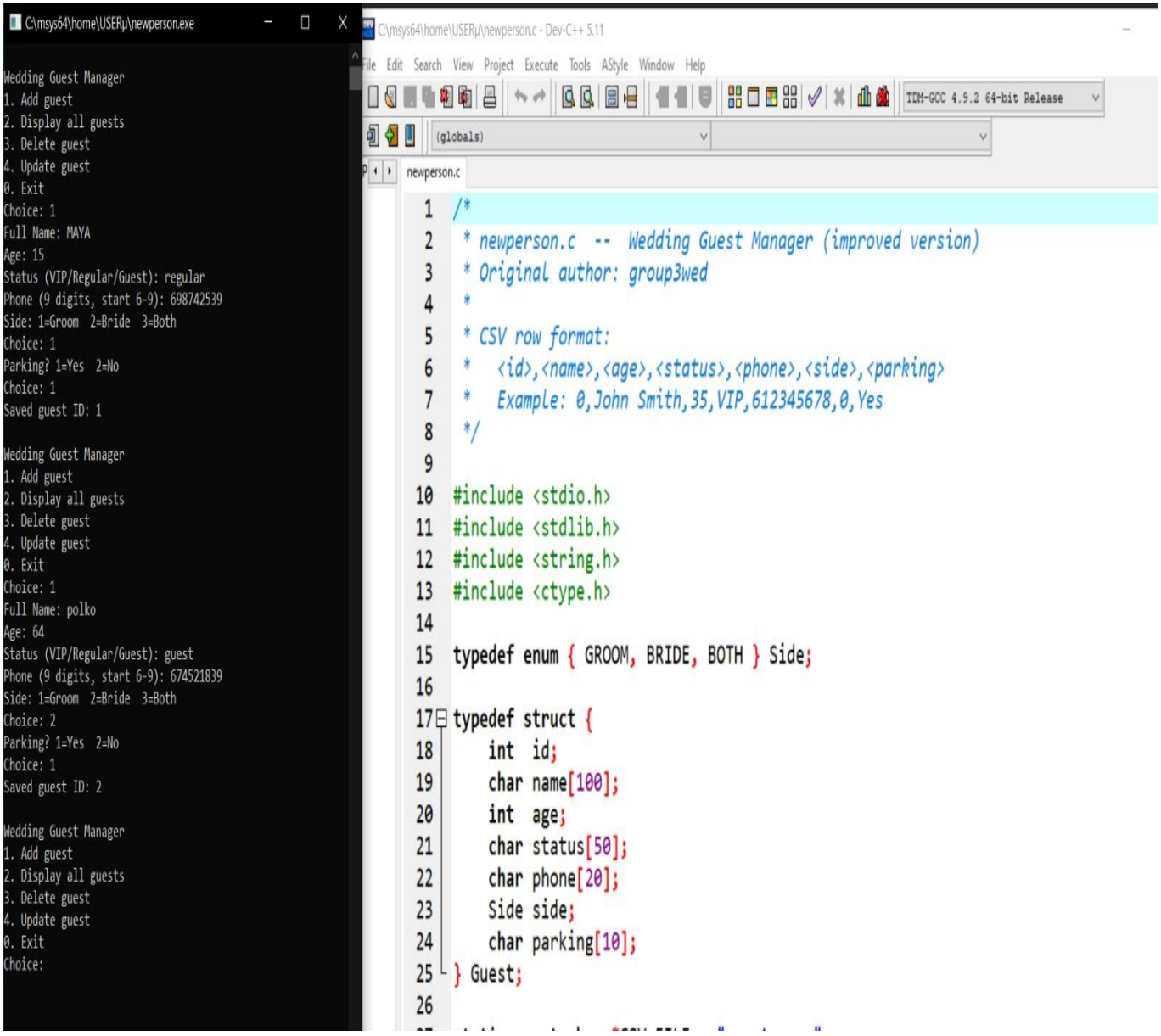
PERSON AND CATEGORY MANAGEMENT: IMPLEMENTATIONS & TESTS

Person Management

This module concerns the registration of the guests at a wedding ceremony, alongside all the operations carried out on them (Entry, Update, Delete). The previous model in the second report being lacking, the following is an update version

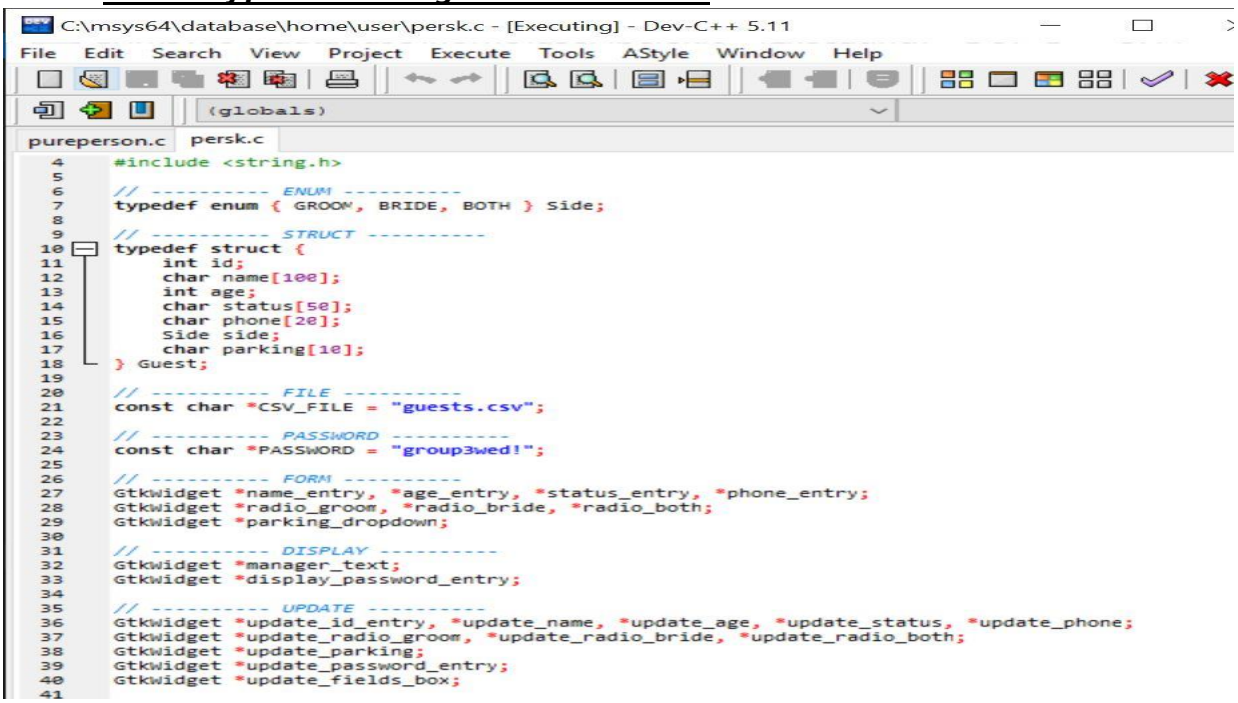
- The c-program was written and thoroughly debugged using dev c++ software

1. *Extract of Person Management C-code*



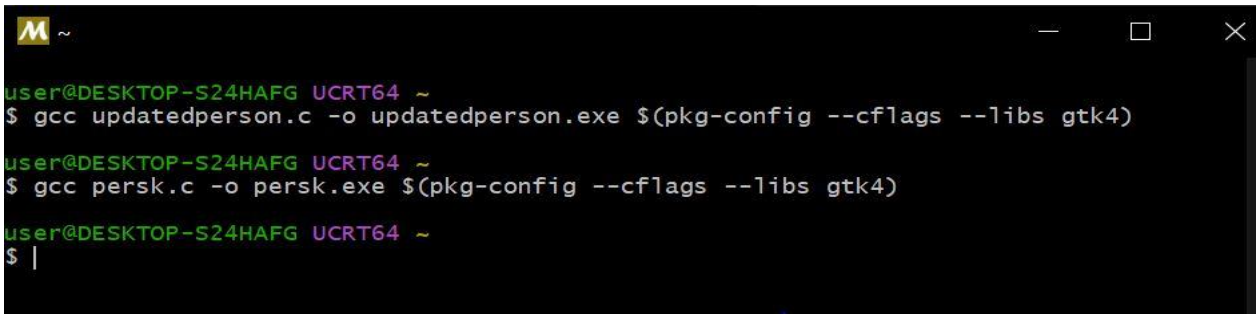
- Once written, the c-program was converted into a GTK form using GTK-specific libraries

2. ***Extract of person management GTK c-code***



- The csv file was automatically generated below the gtk person management file immediately the gtk compiled successfully.

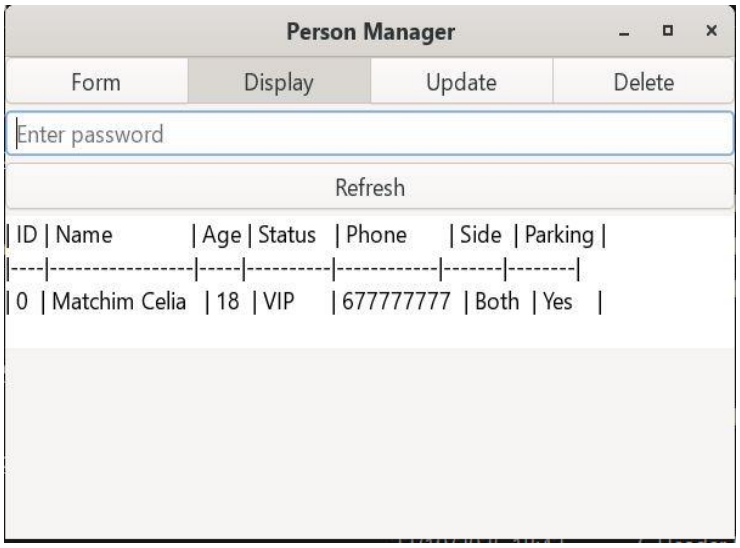
3. ***GTK ran successfully***



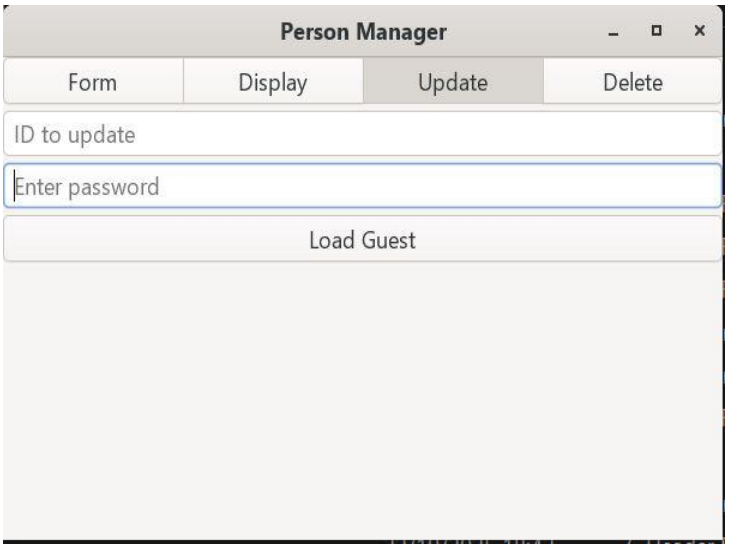
4. ***Csv file***

- Tests were carried out to verify that the Person management form performs its job correctly

(a) Person registration



(b) Person update



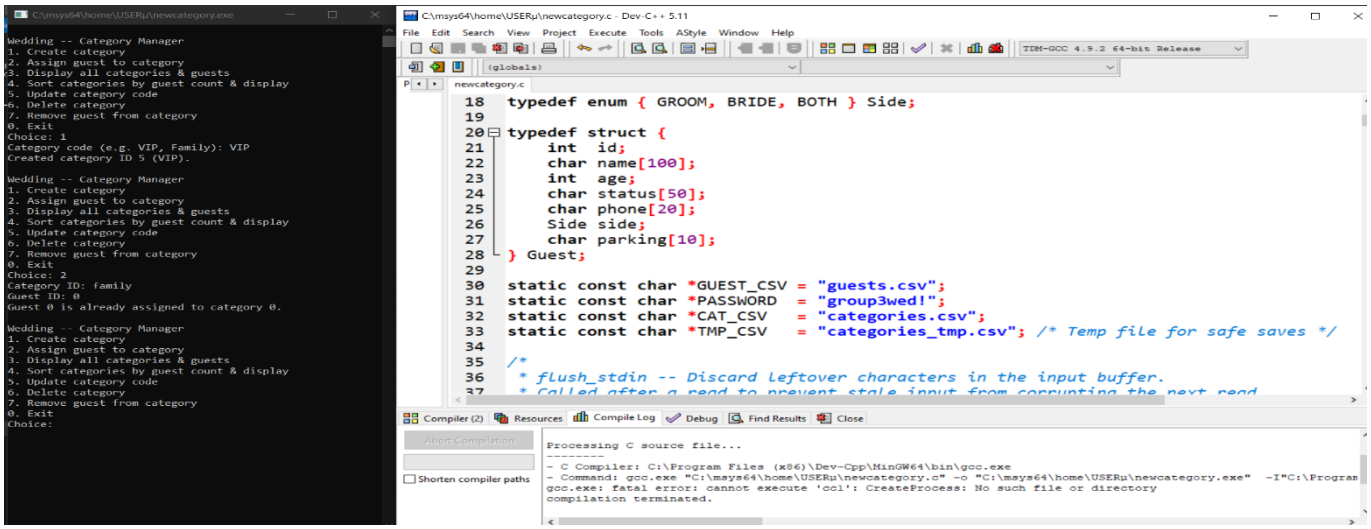
Category Management

The category management module was implemented in order to allow for creation of different categories at a wedding ceremony.

-Just like the person management module, alongside all the operations (Display, Update, Delete, Assign Guests)

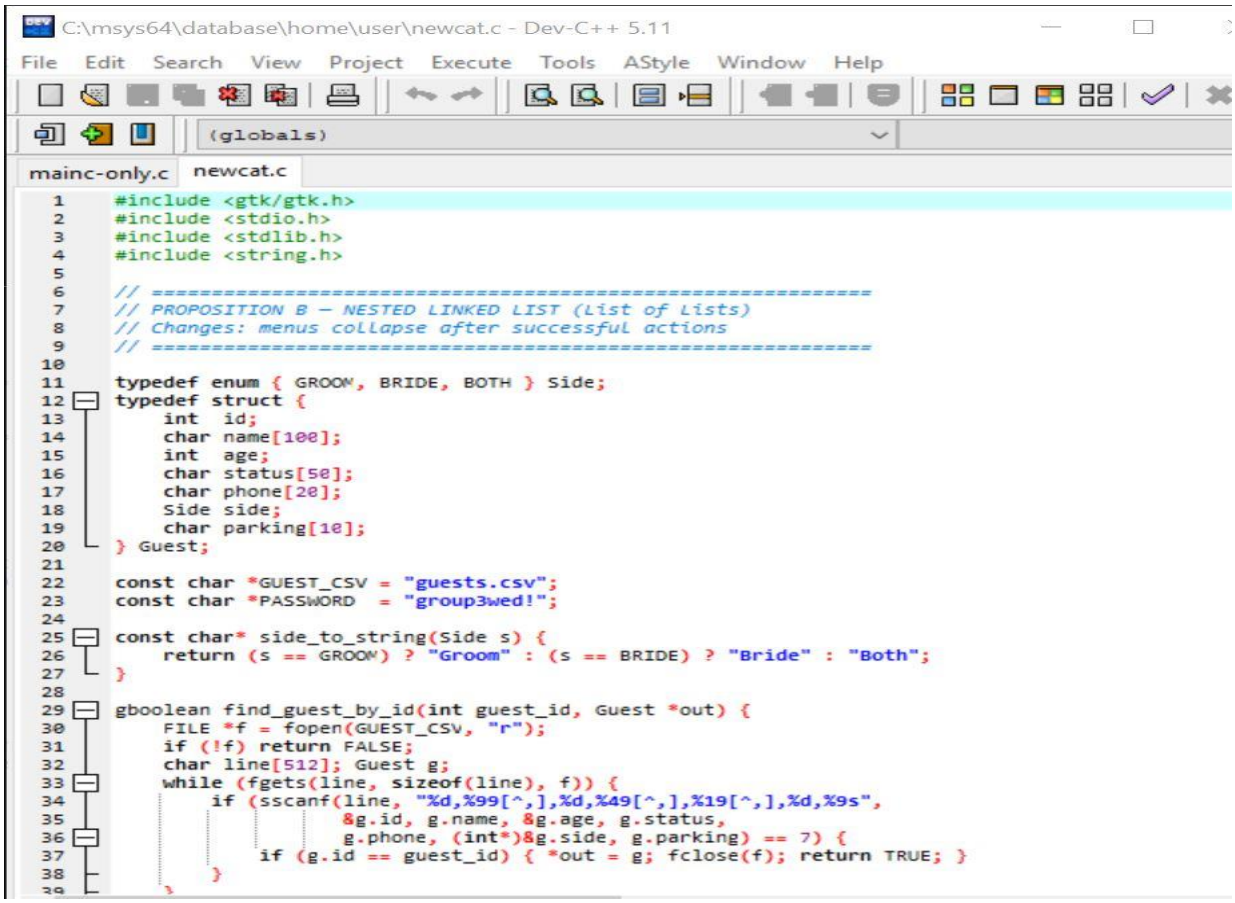
- For assigning guests, the code was linked to the person management code, through the ID
- The code was thoroughly debugged using dev c++ software

5. *Extract of Category Management C-code*



- Once the category management c program was correctly implemented, the GTK form version was created.

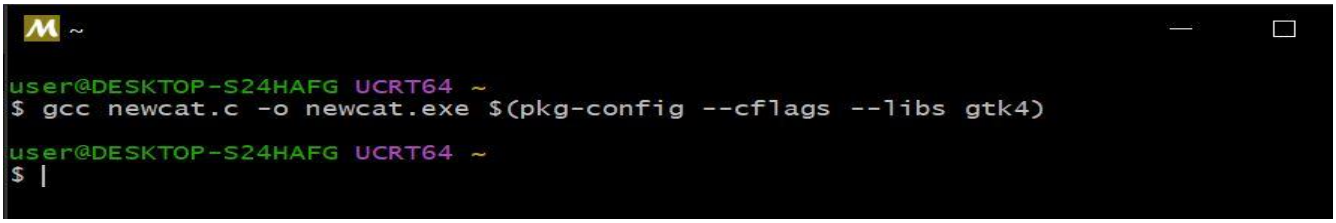
6. ***Extract of Category Management GTK code***



-
-
-

Haven properly run the code, the csv file was created automatically below the category management.

7. ***Category ran successfully***

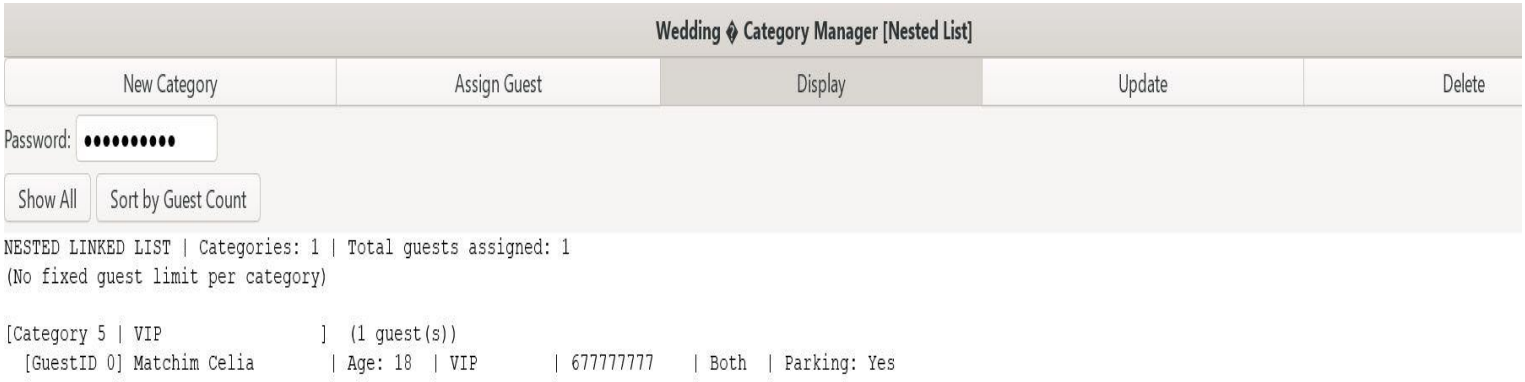


8. ***Csv file***

- Haven properly created the csv file, test were carried out, similarly to those in the person management, to ensure functionability

9. Category Manager Window

10. Assignment Page

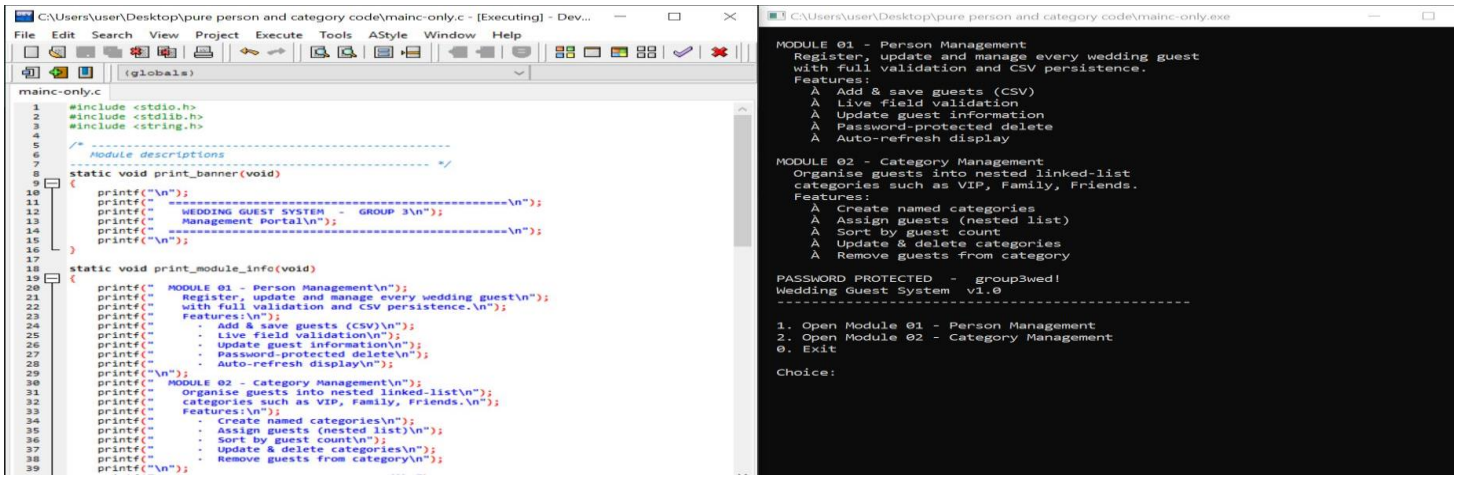


COMBINATION OF PERSON AND CATEGORY MANAGEMENT ONTO A SINGLE WINDOW

Haven made the person and category management fully-functional; it is necessary to combine them in a single window where a user has the possibility of choosing which to fill.

- The c-code, mainapp, was written, thoroughly de bugged and converted to a GTK form, and the csv file generated subsequently

11. Main code with c only



- Css was added to the GTK inorder to beautify the window

12. *Extract of Main code with Css*

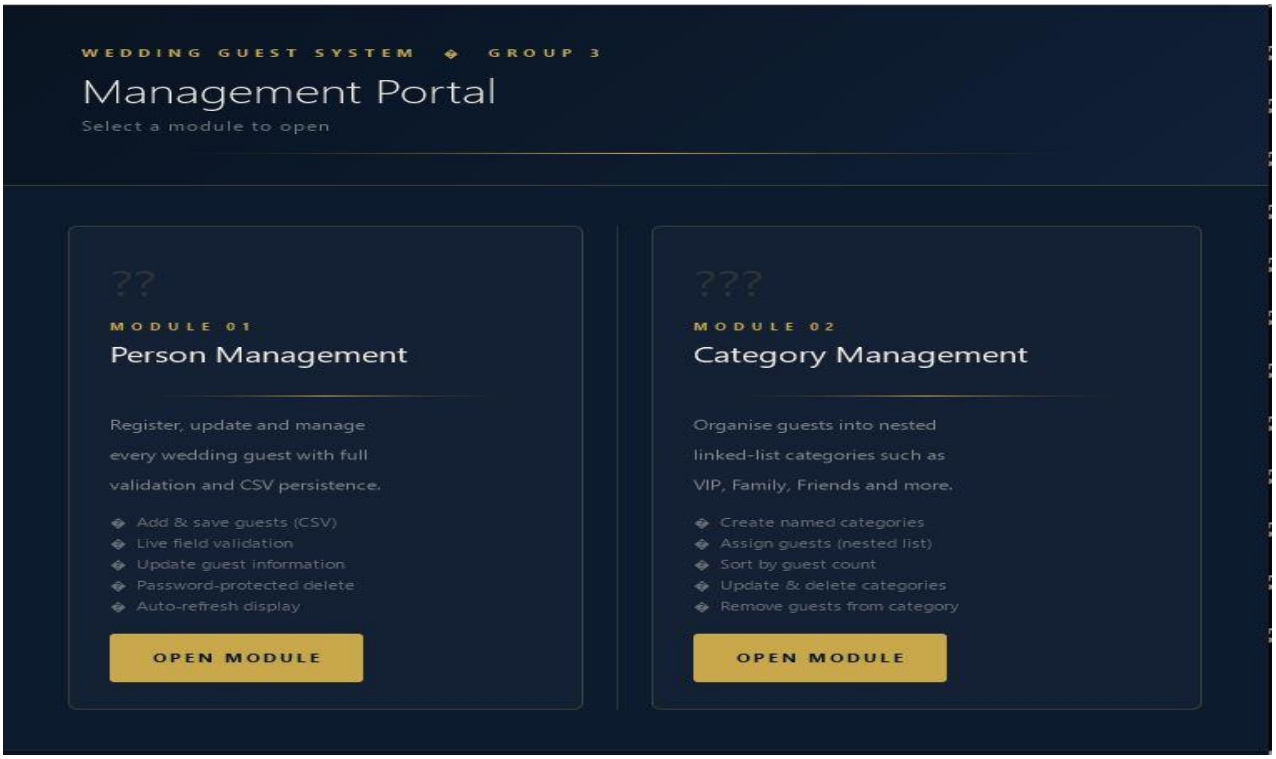
```
#include <gtk/gtk.h>
#include <stdlib.h>
#include <string.h>

/* ------------------------------------------------------
   CSS - dark navy + gold palette
------------------------------------------------------ */
static const char *APP_CSS =
/* Window */
"window {"
"   background-color: #0d1b2e;"
"}"

/* Top banner */
".banner {"
"   background: linear-gradient(135deg, #091422 0%, #0d1b2e 60%, #112038 100%);"
"   padding: 36px 48px 28px 48px;"
"   border-bottom: 1px solid rgba(201,168,76,0.25);"
"}"
".banner-eyebrow {"
"   color: #c9a84c;"
"   font-size: 10px;"
"   letter-spacing: 4px;"
"   font-weight: bold;"
"}"
".banner-title {"
"   color: #f5f0e8;"
"   font-size: 30px;"
"   font-weight: 300;"
"   margin-top: 6px;"
"}"
".banner-sub {"
"   color: rgba(245,240,232,0.45);"
"   font-size: 12px;"
"   margin-top: 4px;"
"   letter-spacing: 1px;"
"}"
".gold-rule {"
"   background: linear-gradient(90deg, transparent, #c9a84c, transparent);"
"   min-height: 1px;"
"   margin: 16px 0 0 0;"
```

13. *mainapp ran Successfully*

```
user@DESKTOP-S24HAFG UCRT64 ~
$ gcc mainapp.c -o mainapp.exe $(pkg-config --cflags --libs gtk4)
mainapp.c: In function 'on_launch_person':
mainapp.c:160:9: warning: 'gtk_message_dialog_new' is deprecated [-Wdeprecated-decla
rations]
  160 |         GtkWidget *dlg = gtk_message_dialog_new(
      |         ^~~~~~~~~
In file included from C:/msys64/database/ucrt64/include/gtk-4.0/gtk/gtk.h:194,
                 from mainapp.c:16:
C:/msys64/database/ucrt64/include/gtk-4.0/gtk/deprecated/gtkmessagedialog.h:81:12: n
ote: declared here
   81 | GtkWidget* gtk_message_dialog_new       (GtkWindow      *parent,
      |            ^~~~~~~~~~~~~~~~~~~~~
mainapp.c: In function 'on_launch_category':
mainapp.c:181:9: warning: 'gtk_message_dialog_new' is deprecated [-Wdeprecated-decla
rations]
  181 |         GtkWidget *dlg = gtk_message_dialog_new(
      |         ^~~~~~~~~
C:/msys64/database/ucrt64/include/gtk-4.0/gtk/deprecated/gtkmessagedialog.h:81:12: n
ote: declared here
   81 | GtkWidget* gtk_message_dialog_new       (GtkWindow      *parent,
      |            ^~~~~~~~~~~~~~~~~~~~~

user@DESKTOP-S24HAFG UCRT64 ~
$
```

14. *Csv file*

15. Main Window



NB:

- A password was used to prevent guests from accessing fields other than registration. Password: group3wed!

-For being lengthy, full codes could not be integrated in our report. Full codes are available in the GitHub repository at this link: https://github.com/celia-pixel-rgb/Project-Wedding-Ceremony
- All the outputs of the codes are present in the repository too.

CONCLUSION

With Modules 1 and 2 successfully implemented, we are ready for the implementation of subsequent modules.